

第27回 バトルを完成させる

今回のゴール

- 第16回から育ててきたバトルの「完成版」を読み、全体の流れを説明できる。
- 「どこを変えると、何が変わるか」を見つけられる。
- `String.format` で、表示を整えられる。
- プレイヤーが攻撃する敵を選べるようにする処理を、自分で完成させられる。

前回とのつながり

第26回までで、パーティー、複数の敵、乱数、入力のエラー対策がそろった。今回は、プレイヤーが攻撃する敵を選べるようにし、表示と結果の出し方を整えて、ゲームとして一通り動く「基準版のバトル」にする。次回からは、この基準版に自分で機能を追加していく。

実習時のフォルダ構成

```
ワークスペース
├─ java27_sample
│   └─ src
│       └─ battle
│           ├─ Character.java  ← 全員に共通（抽象クラス）
│           ├─ Hero.java       ← 勇者（葉草・会心の一撃）
│           ├─ Wizard.java     ← 魔法使い（火の魔法）
│           ├─ Warrior.java    ← 戦士（おの）
│           ├─ Priest.java      ← 僧侶（杖）
│           ├─ Enemy.java      ← 敵（経験値・強攻撃）
│           └─ Main.java       ← バトルの進行
└─ java27_exercise
    └─ src
        └─ battle
            └─ (sample と同じ 7 ファイル)
```

- 配布ファイル: `プログラム/第27回/sample/battle/`、`プログラム/第27回/exercise/battle/`
- 解答例: `プログラム/第27回/answer/battle/`
- `Main.java` 以外は、第26回の解答例と同じ。

講義内容

1. 全体の地図

クラス	役割	主なメソッド
Character	全キャラクターの共通部分。HP のルールを守る	attack (抽象)、normalAttack、damage、heal、isAlive、showStatus
Hero など 4 職業	攻撃のしかたの違い	attack をオーバーライド
Enemy	敵。経験値と強攻撃	attack、getExp
Main	バトルの進行	main、showAll、chooseTarget、enemyTurn、inputNumber

2. 1 ターンの流れ

```
while (味方も敵も残っている)
  └ 2 ターン目なら僧侶が加わる
  └ showAll: 全員の状態を表示
  └ inputNumber: コマンドを選ぶ
  └ switch
    └ 1 攻撃 → chooseTarget で敵を選び、全員で攻撃。たおした敵を enemies から外す
    └ 2 薬草 → hero.useHerb()
    └ 3 にげる → escaped = true
    └ 4 ためる → hero.powerUp(3)
  └ にげた、または敵が全滅 → break
  └ enemyTurn: 敵がランダムに攻撃。たおれた仲間を party から外す
  └ 味方が全滅 → break
  └ turn++
結果を表示
```

3. どこを変えると、何が変わるか

変えたいこと	変える場所
敵の種類・数・強さ	Main の enemies.add(...)
仲間の種類・強さ	Main の party.add(...)、各職業クラスのコンストラクタの引数
ある職業の攻撃のしかた	その職業クラスの attack
HP の上限・下限のルール	Character の damage、heal
コマンドの種類	Main の commands 配列と switch
敵がねらう相手の決め方	Main の enemyTurn
表示のしかた	Character の showStatus、Main の showAll

次回・次々回は、この表を見ながら、自分で機能を追加する。

4. String.format:表示を整える

```
String.format("%dターンで勝利！ 経験値%dを手に入れた", turn, totalExp)
```

書き方	意味
<code>%s</code>	文字列を入れる場所
<code>%d</code>	整数を入れる場所

- `%s`・`%d` の場所に、後ろに書いた値が 順番に 入る。
- `+` でつなぐより、できあがる文の形が読みやすい。
- 値の型と `%` の種類が合わないと、実行時エラーになる(例: `%d` に文字列を渡す)。

5. サンプルを読む

プログラム/第27回/sample/battle/Main.java

```
package battle;

import java.util.ArrayList;
import java.util.InputMismatchException;
import java.util.Random;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Random random = new Random();
        String[] commands = { "攻撃", "薬草を使う", "にげる", "ためる" };

        Hero hero = new Hero("勇者", 30, 8, 2);
        Priest priest = new Priest("僧侶", 25, 4);

        // 味方パーティー
        ArrayList<Character> party = new ArrayList<>();
        party.add(hero);
        party.add(new Wizard("魔法使い", 15, 5));
        party.add(new Warrior("戦士", 40, 10));

        // 敵のグループ
        ArrayList<Enemy> enemies = new ArrayList<>();
        enemies.add(new Enemy("スライム", 20, 5, 5));
        enemies.add(new Enemy("ゴブリン", 40, 7, 12));
        enemies.add(new Enemy("オーク", 70, 9, 30));

        int turn = 1;
        int totalExp = 0;
        boolean escaped = false;

        System.out.println("魔物の群れがあらわれた！");

        // ===== ターンのくり返し =====
    }
}
```

```

while (!party.isEmpty() && !enemies.isEmpty()) {
    System.out.println();
    System.out.println("--- ターン" + turn + " ---");

    if (turn == 2) {
        party.add(priest);
        System.out.println(priest.getName() + "が仲間に加わった！");
    }

    showAll(party, enemies);

    // ----- 味方の行動 -----
    int command = inputNumber(scanner, "コマンドを選ぶ", commands);
    switch (command) {
        case 1:
            Enemy target = chooseTarget(scanner, enemies);
            for (Character member : party) {
                if (enemies.isEmpty()) {
                    break;
                }
                // ねらった敵をたおしていたら、先頭の敵をねらう
                if (!target.isAlive()) {
                    target = enemies.get(0);
                }
                member.attack(target, turn);
                if (!target.isAlive()) {
                    System.out.println(target.getName() + "をたおした！");
                    totalExp += target.getExp();
                    enemies.remove(target);
                }
            }
            break;
        case 2:
            if (hero.isAlive()) {
                hero.useHerb();
            } else {
                System.out.println(hero.getName() + "はたおれていて薬草を使えない！");
            }
            break;
        case 3:
            System.out.println("パーティーはにげだした！");
            escaped = true;
            break;
        case 4:
            hero.powerUp(3);
            System.out.println(hero.getName() + "は力をためた！ 攻撃力が" + hero.getPower() +
                "になった");
            break;
    }
    // にげた、または敵が全滅したら、ここで終わる（turn を増やさない）
    if (escaped || enemies.isEmpty()) {
        break;
    }

    // ----- 敵の行動 -----
    enemyTurn(enemies, party, turn, random);
}

```

```

        if (party.isEmpty()) {
            break;
        }
        turn++;
    }

    // ===== 結果 =====
    System.out.println();
    if (escaped) {
        System.out.println("うまくにげされた。");
    } else if (enemies.isEmpty()) {
        System.out.println(String.format("%dターンで勝利！ 経験値%dを手に入れた", turn, totalExp));
    } else {
        System.out.println(String.format("%dターン目にパーティーは全滅した...", turn));
    }
}

// 味方と敵の全員の状態を表示する（敵には番号を付ける）
static void showAll(ArrayList<Character> party, ArrayList<Enemy> enemies) {
    for (Character member : party) {
        member.showStatus();
    }
    System.out.println("敵：" + enemies.size() + "体");
    for (int i = 0; i < enemies.size(); i++) {
        Enemy enemy = enemies.get(i);
        System.out.println(String.format(" %d: %s HP:%d/%d", i + 1, enemy.getName(),
            enemy.getHp(), enemy.getMaxHp()));
    }
}

// ねらう敵を番号で選ばせる
static Enemy chooseTarget(Scanner scanner, ArrayList<Enemy> enemies) {
    String[] names = new String[enemies.size()];
    for (int i = 0; i < enemies.size(); i++) {
        names[i] = enemies.get(i).getName();
    }
    int number = inputNumber(scanner, "ねらう敵を選ぶ", names);
    return enemies.get(number - 1);
}

// 残っている敵が、1 体ずつランダムに相手を選んで攻撃する
static void enemyTurn(ArrayList<Enemy> enemies, ArrayList<Character> party, int turn, Random
    random) {
    for (Enemy enemy : enemies) {
        if (party.isEmpty()) {
            break;
        }
        Character target = party.get(random.nextInt(party.size()));
        enemy.attack(target, turn);
        if (!target.isAlive()) {
            System.out.println(target.getName() + "はたおれた...");
            party.remove(target);
        }
    }
}
}

```

```
// 選択肢を表示し、1～選択肢の数 の番号が入力されるまで聞いて、その番号を返す
static int inputNumber(Scanner scanner, String message, String[] choices) {
    for (int i = 0; i < choices.length; i++) {
        System.out.println((i + 1) + ": " + choices[i]);
    }
    int number = 0;
    do {
        System.out.print(message + " > ");
        try {
            number = scanner.nextInt();
        } catch (InputMismatchException e) {
            System.out.println("数字を入力してください");
            scanner.next();
        }
    } while (number < 1 || number > choices.length);
    return number;
}
}
```

- 第26回の `inputCommand` を、`inputNumber` に名前を変えて、コマンドの選択にも敵の選択にも使えるようにした。メッセージと選択肢を引数で受け取るので、同じメソッドを 2 か所で使い回せる。
- `chooseTarget` は、敵の名前を入れた配列を作ってから `inputNumber` に渡している。画面の番号は 1 から、リストの番号は 0 からなので、`number - 1` 番目を返す。
- `main` の中身は、「何をするか」の流れが中心になり、細かい処理はメソッドに分かれている(第18回の考え方)。
- 敵が全滅したときは、`turn++` の前に `break` している。ここで `break` しないと、勝利したターン数が 1 つ 多く表示されてしまう。

6. 実行する前に予想する

- コマンドを選ぶ > 1 の後、ねらう敵を選ぶ > 2 と入力すると、誰を攻撃するか。
- ゴブリンをねらっている途中でゴブリンをたおしたら、残りのメンバーは誰を攻撃するか(`case 1` を読む)。

7. 実行結果の例

最初の部分は、毎回同じになる(敵の攻撃は乱数で変わる)。

魔物の群れがあらわれた！

--- ターン1 ---

勇者 HP:30/30

魔法使い HP:15/15

戦士 HP:40/40

敵：3体

1: スライム HP:20/20

2: ゴブリン HP:40/40

3: オーク HP:70/70

1: 攻撃

2: 薬草を使う

3: にげる

4: ためる

コマンドを選ぶ > 1

1: スライム

2: ゴブリン

3: オーク

ねらう敵を選ぶ > 2

勇者の攻撃！ ゴブリンに8のダメージ

魔法使いは火の魔法をとねえた！ ゴブリンに10のダメージ

戦士のおのの一撃！ ゴブリンに10のダメージ

(最後の例)

5ターンで勝利！ 経験値47を手に入れた

8. エラーを体験する

1. `chooseTarget` の `return enemies.get(number - 1);` を `return enemies.get(number);` に変えて保存する。
2. 実行して、ねらう敵に **最後の番号**(1 ターン目なら `3`)を選ぶ。 `IndexOutOfBoundsException` になることを確認する。
3. 最後以外の番号を選んだときは、エラーにならずに **となりの敵** を攻撃してしまう(論理エラー)。どちらも同じ 1 か所のまちがいから起きている。確認したら元に戻す。

演習1 サンプルを実行する

`java27_sample` でサンプルを実行し、勝つまで、または全滅するまで遊ぶ。「にげる」も試す。

演習2 基準版を完成させる

`java27_exercise` を作り、`battle` パッケージに `プログラム/第27回/exercise/battle/` の 7 ファイルを入れる。

演習2-1 ねらう敵を選べるようにする

- 今の `chooseTarget` は、いつも先頭の敵を返している。実行して、どの敵もねらえないことを確認する。
- 敵の名前を入れた `String` 型の配列を作り、`inputNumber` で番号を選ばせて、選ばれた敵を返すようにする。

ヒント:

- 配列の大きさは `enemies.size()`。`new String[enemies.size()]` で作れる(第22回)。
- `for` 文で、`names[i] = enemies.get(i).getName();` と入れていく。
- 返すのは `enemies.get(number - 1)`。

演習2-2 勝利メッセージを整える

- 勝ったときの表示を、`String.format` を使って `5ターンで勝利！ 経験値47を手に入れた` の形にする。

演習2-3 ピンチの表示

- `showAll` で、仲間の HP が最大 HP の 4 分の 1 以下なら、その仲間の表示の次の行に `↑ ピンチ！` と表示する。

ヒント: `member.getHp() <= member.getMaxHp() / 4`

完成条件:

- ねらう敵を番号で選べる。範囲外の番号や数字以外を入れても、聞き直される。
- 勝ったときに、ターン数と経験値が表示される。
- HP が少ない仲間に `↑ ピンチ！` が付く。

```
ねらう敵を選ぶ > 2
勇者の攻撃！ ゴブリンに8のダメージ
```

発展(任意)

- 敵の数値(HP・攻撃力・経験値)を変えて、勝ったり負けたりするちょうどよい強さに調整する。
- 戦闘の最後に、生き残った仲間の名前を一覧で表示する。

確認問題

問1 出力は何か。

```
System.out.println(String.format("%sのHPは%dです", "勇者", 16));
```

問2 出力は何か。

```
ArrayList<String> list = new ArrayList<>();
list.add("スライム");
list.add("ゴブリン");
for (int i = 0; i < list.size(); i++) {
    System.out.println((i + 1) + ":" + list.get(i));
}
```

問3 出力は何か。

```
public static void main(String[] args) {
    int[] exp = { 5, 12, 30 };
    System.out.println(sum(exp));
}

static int sum(int[] values) {
    int total = 0;
    for (int v : values) {
        total += v;
    }
    return total;
}
```

■ 答え

- 問1: 勇者のHPは16です
- 問2: 1:スライム と 2:ゴブリン (表示の番号は `i + 1`、取り出す番号は `i`)
- 問3: 47

振り返り

- 1 ターンの流れを、上の図を見ずに説明できるか。
- 「どこを変えると何がかわるか」の表のうち、自分で変えてみたものはどれか。
- `inputNumber` を 2 か所で使い回している理由を説明できるか。

次回へのつながり

次回は、この基準版に「回復魔法」という新しい能力を追加する。僧侶だけでなく、敵の中にも回復できるものを作りたい。職業の親子関係(継承)とは別に、「回復できる」という能力を表す方法を学ぶ。